

Padrões e Princípios na Programação Orientada a Objetos

Jhony Luiz de Almeida

Relembrando (de novo), o que é OO?

- Como visto na primeira aula, orientação a objetos é um paradigma de programação, ou seja, uma forma de enxergar e estruturar um software
 - Isto afeta não só o código, mas também as etapas anteriores à própria programação, como a modelagem do sistema
- Diferentemente dos paradigmas procedural e estruturado, na OO cada pedaço do sistema é modelado como um objeto que possui sua interface pública, e seus detalhes de implementação, que ficam encapsulados

Relembrando, o que é OO?

- Vimos até aqui os pilares base da OO:
 - Abstração
 - Encapsulamento
 - Herança
 - Polimorfismo
- Entretanto, desde sua concepção e ascensão, o paradigma da programação orientada a objetos recebeu diversas contribuições de importantes autores, que sugeriram vários padrões construídos sobre estes pilares

Coesão X Acoplamento

- Coesão diz respeito ao quanto uma classe possui responsabilidades bem definidas e possui baixo nível de acoplamento com muitas classes
- Acoplamento se refere ao quanto uma classe depende diretamente de outras classes, de forma que pode ser afetada por mudanças nelas
 - Herança é um exemplo de relação forte entre classes, onde alterações na classe pai afetam diretamente as classes filhas

Padrões em software

- Padrões em geral visam facilitar a manutenção, extensão e testabilidade de um software. Em outras palavras, em sistemas OO, eles visam maximizar a coesão e reduzir o acoplamento entre as classes e pacotes
- Costumam tornar o código mais legível e familiar aos desenvolvedores, facilitando a leitura e reduzindo as chances de se introduzir bugs acidentalmente
 - Geralmente desenvolvedores gastam mais tempo lendo código existente do que efetivamente escrevendo

Padrões de Projeto (Design Patterns)

- Descritos pela primeira vez no livro *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), escrito pela “Gang of Four” (GoF), composta por Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides
- São 23 padrões ao todo, divididos em 3 categorias
- Cada padrão expõe o contexto de um problema, e uma proposta de solução

Padrões de Projeto (Design Patterns)

- **Criacional:** padrões que definem formas de criar e controlar a criação de novas instâncias de objetos
 - Pode por exemplo limitar a quantidade de instâncias
- **Estrutural:** padrões que definem como combinar classes e objetos para formar estruturas mais complexas
 - Oferecem estruturas comuns para resolver problemas similares
- **Comportamental:** padrões que definem como certos objetos interagem para atingir um objetivo final
 - Geralmente se utilizam fortemente do polimorfismo

Padrões de Projeto (Design Patterns)

C Abstract Factory

S Adapter

S Bridge

C Builder

B Chain of Responsibility

B Command

S Composite

S Decorator

S Facade

C Factory Method

S Flyweight

B Interpreter

B Iterator

B Mediator

B Memento

C Prototype

S Proxy

B Observer

C Singleton

B State

B Strategy

B Template Method

B Visitor

Padrões Criacionais

- **Singleton**
- **Builder**
- **Factory method**
- **Abstract factory**
- **Prototype**

Padrões Estruturais

- **Proxy**
- **Bridge**
- **Adapter**
- **Decorator**
- **Composite**
- **Facade**
- **Flyweight**

Padrões Comportamentais

- **Strategy**
- **State**
- **Chain of responsibility**
- **Command**
- **Observer**
- **Iterator**
- **Template method**
- **Visitor**
- **Mediator**
- **Memento**
- **Interpreter**

SOLID

- Sigla concebida pelo autor Robert C. Martin (“Uncle Bob”) para representar 5 princípios de design de código orientado a objetos
 - **Single Responsibility:** responsabilidade única
 - **Open/Closed:** princípio do aberto/fechado
 - **Liskov Substitution:** substituição de Liskov
 - **Interface Segregation:** segregação de interfaces
 - **Dependency Inversion:** inversão de dependências

SOLID

- ❑ **Responsabilidade única:** classes devem ter apenas um motivo para mudar
- ❑ **Aberto/Fechado:** classes devem ser abertas à extensão, mas fechadas para modificações
- ❑ **Substituição de Liskov:** classes filhas devem ser substituíveis por suas superclasses (e vice-versa)
- ❑ **Segregação de interfaces:** é melhor ter várias pequenas interfaces específicas que uma interface grande que agregue muitas funções
- ❑ **Inversão de dependência:** módulos de alto nível não devem depender de módulos de baixo nível, ambos devem depender de abstrações

Padrões de Arquitetura do Código

- **Camadas:** um conceito comum em padrões de arquitetura, consiste em dividir a aplicação em camadas com responsabilidades específicas
 - Um exemplo popular é a arquitetura MVC (Model-View-Controller), muito usada em aplicações web
- **Hexagonal:** usa o conceito de portas e adaptadores para isolar o núcleo de um módulo das suas dependências de entrada e saída de dados
 - Cada porta é efetivamente uma interface, enquanto os adaptadores são suas implementações

Padrões de Arquitetura de Sistemas

- **Monólitos:** aplicações que abrangem vários domínios de negócio e funcionalidades, que podem estar estruturados de forma modular ou não
- **Microserviços:** aplicações que abrangem um único domínio e podem se comunicar com outros microserviços para desempenhar certas tarefas

Frameworks

- São conjuntos de componentes de software reutilizáveis, similar a bibliotecas, mas que afetam à aplicação ou um segmento dela de modo amplo, afetando diretamente como o código é escrito e estruturado
- Ao custo de impor certa estrutura e/ou regras a seguir, oferece diversas facilidades para acelerar o desenvolvimento
- O framework Spring, por exemplo, pode automatizar toda a criação dos objetos que compõe a aplicação, por meio da funcionalidade de **injeção de dependências**

Mapeamento Objeto-Relacional (ORM)

- Classe de frameworks para auxiliar a persistir objetos em um banco relacional (SQL), como MySQL ou Postgres
- **DataMapper:** objetos tem um mapeamento explícito e normalmente há uma divisão entre a classe responsável por realizar as operações e a classe que representa a entidade a ser persistida.
- **ActiveRecord:** a própria classe da entidade persistida possui os métodos para realizar as operações no banco de dados, como se a classe fosse a própria tabela do banco